



Iby and Aladar Fleischman
Faculty of Engineering
Tel Aviv University

הפקולטה להנדסה
ע"ש איבי ואלדר פליישרמן
אוניברסיטת תל-אביב

Test Board for VLSI - single neuron hardware accelerator engine

Project Number: 24-1-1-3039

Project Report

Student: Yuval Desalto ID: 209544824

Student: Ariel Richter ID: 313285611

Supervisor: Zeev Bikowsky

Project Carried Out at: Tel Aviv University

Contents

1	Abstract.....	3
2	Introduction.....	4
3	Theoretical background.....	5
4	Architecture & Implementation	6
4.1	Hardware Description	7
4.2	Software Description.....	9
5	RTL structure and system integration.....	10
5.1	System Workflow Overview	11
6	Test execution and Validation results	12
6.1	Test Duration.....	12
6.2	Execution of Multiple Tests	13
6.3	Failure Indication on 7-Segment Display.....	14
6.4	Quantitative Requirement #1: Coverage.....	15
6.4.1	Data Coverage	15
6.4.2	FSM coverage.....	19
6.4.3	Toggle coverage	21
6.5	Quantitative Requirement #2: Clock Frequency	21
6.6	Quantitative Requirement #3: Power Consumption	23
7	Post silicone Validation Planning	24
8	Evaluation and future work.....	26
8.1	Difficulties and Challenges.....	26
8.2	Suggestions for Improvement and Future Work	27
8.3	Lessons Learned.....	28
9	Project Documentation.....	29
10	References.....	30

List of figures

Figure 1	- System-Level Block Diagram of the FPGA-Based Test Environment.....	3
Figure 2	- System Architecture Block Diagram (FPGA Implementation)	6
Figure 3	- DE10-Lite Board and Physical Interfaces.....	7
Figure 4	- Internal Block Diagram of the DE10-Lite Board.....	8

Figure 5 - Saleae Logic Analyzer Connected to the DE10-Lite Board	8
Figure 6 - RTL Block Diagram Generated by Quartus.....	10
Figure 7 - Single Test Duration in Continuous Execution (Test $n > 1$).....	12
Figure 8 - First Test Execution Timing.....	13
Figure 9 - Logic Analyzer Capture Showing 44 Consecutive Tests.....	14
Figure 10 - Failure Display Showing Test Indices 290 and 256.....	14
Figure 11- Delayed Result Capture Across Multiple Tests	16
Figure 12 - Validator FSM State Encoding while executing delayed result tests	16
Figure 13 - FSM State Capture in the Validator During Delayed Read	17
Figure 14 - chip_sel Drop and Correct Test Restart	18
Figure 15 - non-continuous wr_en Pulses During Data Streaming	19
Figure 16 - FSM State Capture During Single Test Execution	20
Figure 17 - Control Unit FSM State Definition	20
Figure 18 - Control Unit Synthesized FSM Structure	20
Figure 19 - Generator FSM State Definition	20
Figure 20 - Generator Synthesized FSM Structure.....	20
Figure 21 - Validator FSM State Definition	20
Figure 22 - Validator Synthesized FSM Structure.....	20
Figure 23 - Confidence Metrics from Quartus Power Analyzer showing toggle coverage	21
Figure 24 - Quartus Fmax Summary at 0°C (Typical Voltage).....	22
Figure 25 - Quartus Fmax Summary at 85°C (Worst-Case Operating Conditions)	22
Figure 26 - Power Analyzer report showing total power dissipation	23
Figure 27 - Physical Integration Schematic for Post-Silicon Validation.....	24

List of equations

Equation 1	5
Equation 2	5

1 Abstract

This project presents the design, implementation, and verification of a hardware validation system for a single-neuron MAC (Multiply-Accumulate) Core accelerator, based on the perceptron model. The goal of the project is to develop a comprehensive test environment that enables full functional validation of the MAC Core using FPGA based pre-silicon testing. The system ensures correctness, performance, and robustness across 500 distinct test cases generated by a custom Python script.

The FPGA-based platform integrates a test generator, validator, control unit, and on-chip memory, which together emulate realistic operating conditions for the DUT (Device Under Test). The MAC Core receives a series of pixel-weight pairs and a threshold, performs the MAC computation, applies a binary activation function, and returns a single-bit result. The system compares this result against the expected output and reports failures via 7-segment displays on the DE10-Lite board (as explained in section 5.1).

The test environment supports semi-automated testing, error detection, and interruption handling, ensuring high coverage through both randomized and directed tests. The entire testing process is conducted within the FPGA, without reliance on external memory, providing a fast, accurate, and scalable validation framework.

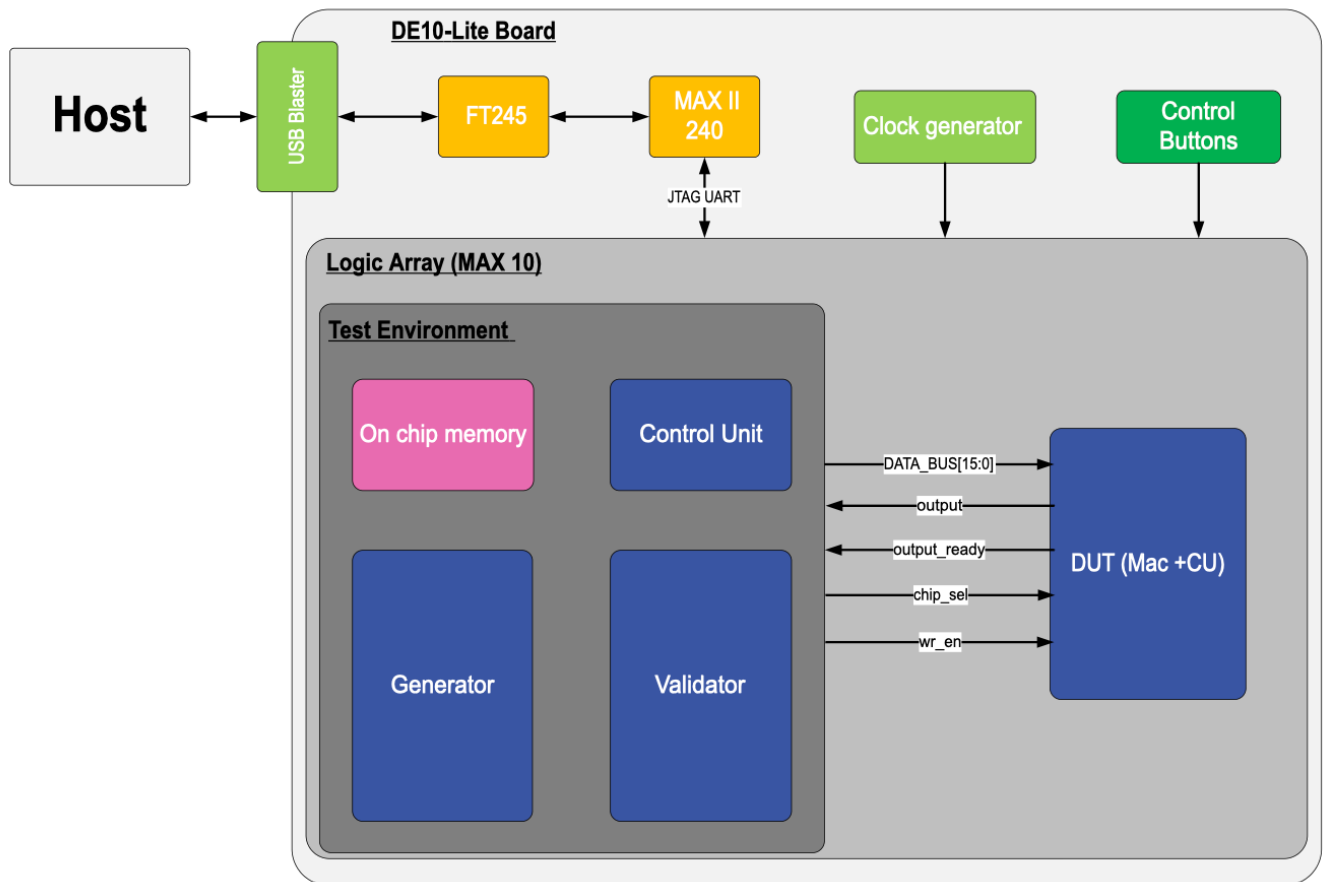


Figure 1 - System-Level Block Diagram of the FPGA-Based Test Environment

2 Introduction

The objective of this project is to implement a full-cycle validation environment for a single-neuron MAC (Multiply-Accumulate) Core accelerator, based on the perceptron model. The MAC Core mimics a single neuron by performing a weighted sum of inputs and applying a binary threshold activation (as explained in chapter 3). The project validates this hardware module at the pre-silicon stage using an FPGA-based test environment.

The motivation behind this work stems from the growing importance of hardware accelerators in modern AI applications, particularly those involving edge devices where efficiency, reliability, and compact design are crucial. Testing such hardware components thoroughly, before manufacturing, can significantly reduce cost and time-to-market. Using an FPGA as a validation platform enables real-time, cycle-accurate testing and provides deep observability into the behavior of the system under test.

To address the validation challenge, we implemented a modular testing system entirely on an FPGA (Intel MAX 10, DE10-Lite board). The system includes a test generator, a control unit, an output validator, on-chip memory to store test vectors and expected results, and a user interface via 7-segment displays. A Python-based test generation script creates 500 diverse test cases, covering both normal and edge case scenarios, which are then executed in real time on the FPGA.

This project was conducted as a collaboration between two development teams. One team was responsible for designing and implementing the MAC Core itself, preparing the final design for tape-out. Our team focused on building the pre-silicon validation infrastructure, ensuring that the MAC Core operates correctly before physical fabrication. This division of responsibilities allowed for parallel development and ensured that the hardware would be rigorously tested under realistic and diverse conditions prior to manufacturing.

In addition, the project was conducted under the guidance of Apple, who both funded the development and provided professional feedback throughout. Midway through the process, we presented our system to Apple engineers in a formal review, receiving constructive insights that directly influenced architectural and verification decisions.

Compared to conventional software-based testbenches, our FPGA-based validation provides significantly greater realism and reliability. Simulated environments often abstract away timing behavior and fail to capture hardware-specific issues such as metastability, signal contention, or asynchronous resets. These limitations can hide critical bugs that only surface after fabrication.

By executing tests under real clock cycles and hardware constraints, our system exposes such issues early and enables accurate validation of control and data paths. Embedding the entire test process within the FPGA eliminates simulation artifacts and ensures deterministic, cycle-accurate results. This approach provides much higher confidence in the MAC Core's correctness before tape-out. Without such validation, issues might remain hidden until post-silicon, risking delays and added cost. Our system addresses this by enabling fast, automated, and cycle-accurate testing that improves pre-fabrication assurance.

3 Theoretical background

Hardware validation platforms are essential in the development and verification of modern integrated circuits, especially in the context of deep learning accelerators and neural network based architectures. This project centers around the construction of a dedicated test system for a single-neuron MAC (Multiply-Accumulate) Core, leveraging the fundamental principles of the perceptron model.

Building such a test system requires an understanding of data processing architectures, control signal synchronization, and real-time validation mechanisms. FPGAs (Field-Programmable Gate Arrays) are particularly well-suited for this task due to their reconfigurability and ability to emulate hardware behavior at the register-transfer level. Unlike pure software simulations, FPGA-based validation enables timing-accurate testing, hardware-constrained signal flow, and direct observation of DUT responses under real operating conditions.

Through the use of a fully synthesized RTL testbench implemented inside the FPGA, the system allows for precise management of test input delivery, output comparison, and error reporting. This method provides clear benefits over traditional software-only environments by identifying functional and integration errors at an earlier stage, reducing post-fabrication debugging effort and time.

The Perceptron Model

The perceptron is a foundational concept in machine learning, mimicking the behavior of a single neuron in a neural network. It operates by performing a linear combination of input values, weighted by their respective coefficients, and adding a bias term. Mathematically, this can be expressed as:

$$y = \sum_{i=1}^n w_i \cdot x_i + b$$

Equation 1

where w_i are the weights, x_i are the inputs, and b is the bias term.

The perceptron's output is determined by an activation function, which applies a threshold to the result of the linear summation. The function is defined as:

$$y = \begin{cases} 1 & ; \quad \text{if } y \geq \text{Threshold} \\ 0 & ; \quad \text{otherwise} \end{cases}$$

Equation 2

In this project, the perceptron acts as the computational core within the MAC block, processing 64 pairs of input features and weights. The output is a single binary decision bit, indicating whether the weighted sum exceeds a specified threshold, as expected in classification or pattern recognition tasks.

4 Architecture & Implementation

The system architecture was carefully designed to support efficient, accurate, and scalable validation of a custom single-neuron MAC Core intended for neural processing applications. The validation is performed at the pre-silicon stage, where the MAC Core is still under development and implemented on the same FPGA device used for testing. This constraint required a compact, modular, and fully self-contained design that could operate entirely on-chip, without relying on external memory.

To address this, the system adopts a hardware testing approach implemented entirely on an FPGA platform. It includes all the logic necessary to generate test vectors, apply them to the Device Under Test (DUT), capture its output, and perform real-time validation. The architecture enables sequential execution of 500 independent test cases, each consisting of 64 pixel-weight pairs and a 22 bit threshold. The DUT's binary output is compared against preloaded expected results, and any mismatch is immediately detected and reported. A built-in failure tracking mechanism and visual interface ensure test transparency and allow users to resume testing after interruptions.

The system is composed of several functional blocks, including a test generator, control unit, validator, memory module, user interface elements (buttons and 7-segment displays), and the Device Under Test (DUT). Each block performs a dedicated role within the validation pipeline, and the architecture reflects a clear separation between control logic, data handling, and output reporting.

The following diagram presents a complete RTL level view of the test system as implemented on the FPGA:

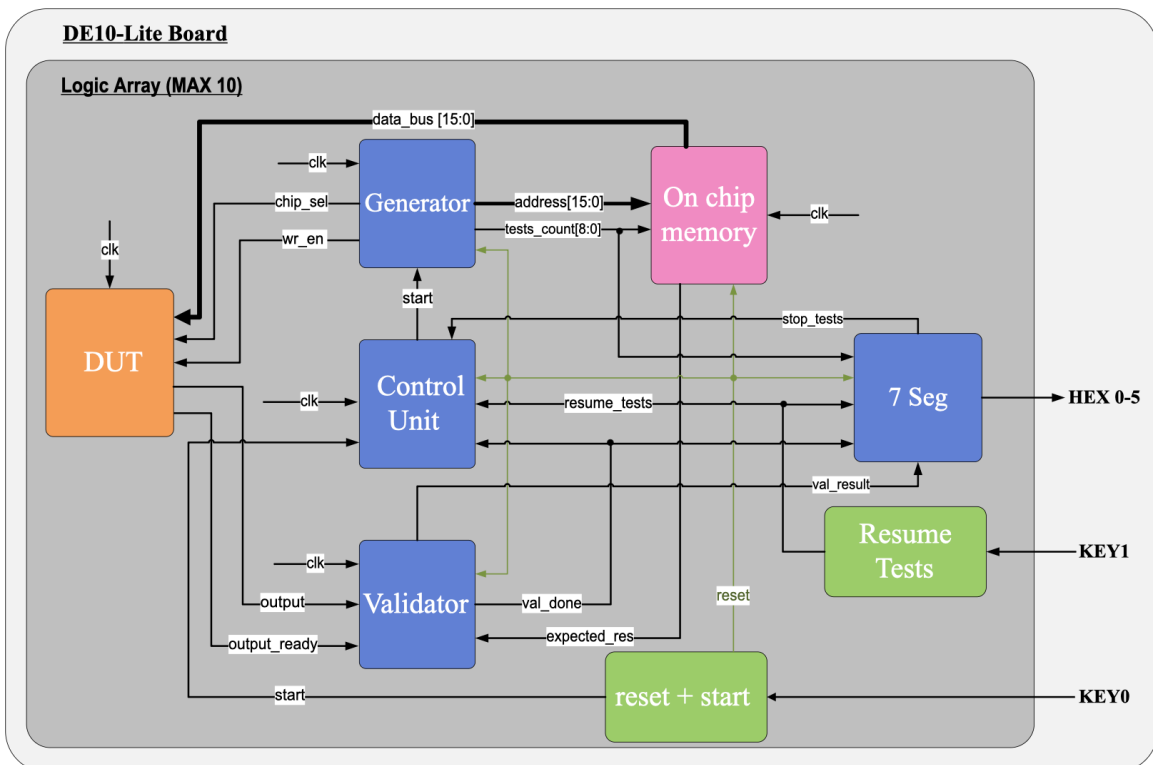


Figure 2 - System Architecture Block Diagram (FPGA Implementation)

4.1 Hardware Description

The hardware platform used in this project is the DE10-Lite FPGA development board by Terasic, based on the Intel MAX 10 device (10M50DAF484C7G). The board includes several integrated components that were directly utilized in the implementation and validation of the system.

FPGA board

The MAX 10 FPGA was used to implement the entire validation system, including the Device Under Test (DUT), the test generator, validator, control logic, and internal memory blocks. The following onboard components were actively used:

- On-Chip Clock Generator - Provided the global 50 MHz system clock. This ensured consistent synchronization across all modules and accurate timing behavior during test execution.
- Push Buttons (KEY0, KEY1) - Used to control test execution:
 - KEY0: triggers system reset and starts the test sequence.
 - KEY1: resumes test execution after two test failures.
- 7-Segment Displays (HEX0-HEX5) - Used to display the indices of the first two failing tests for immediate user feedback.
- GPIO Pins - Utilized to expose internal control signals (chip_sel, output_ready, etc.) for real-time observation via an external logic analyzer.
- User LEDs - Used temporarily for debugging purposes to monitor control or output signals.

The system was designed to be fully self-contained, using only the FPGA's internal memory (on-chip RAM) to store 500 test cases and their expected results, eliminating the need for external SDRAM or host communication during runtime.

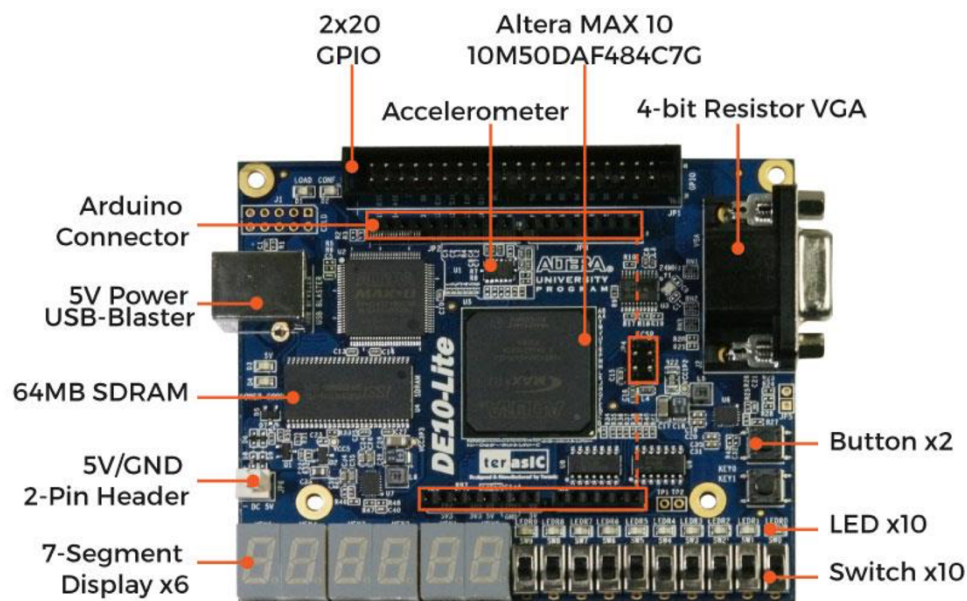


Figure 3 - DE10-Lite Board and Physical Interfaces

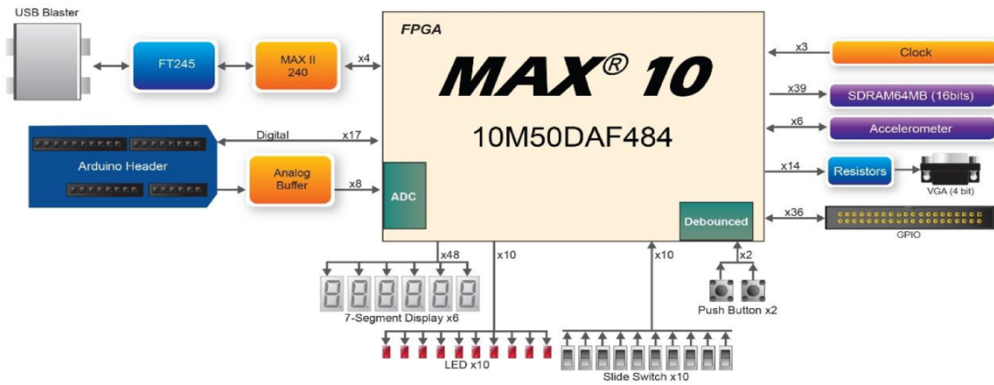


Figure 4 - Internal Block Diagram of the DE10-Lite Board

These two diagrams provide a complete overview of both the physical layout and logical structure of the board, helping to visualize how the components used in the project are mapped to available hardware resources.

Logic Analyzer

While the internal block diagrams and board schematics offer a static view of the hardware resources, they do not capture the system's dynamic behavior during runtime. To bridge this gap and gain deeper insight into the operation of the system in real conditions, we incorporated an external Logic Analyzer as part of the hardware validation setup.

Saleae Logic Analyzer was used extensively throughout the development and testing process. This USB-powered device was connected to the DE10-Lite board via the GPIO header, enabling real-time signal capture from various internal points in the system.

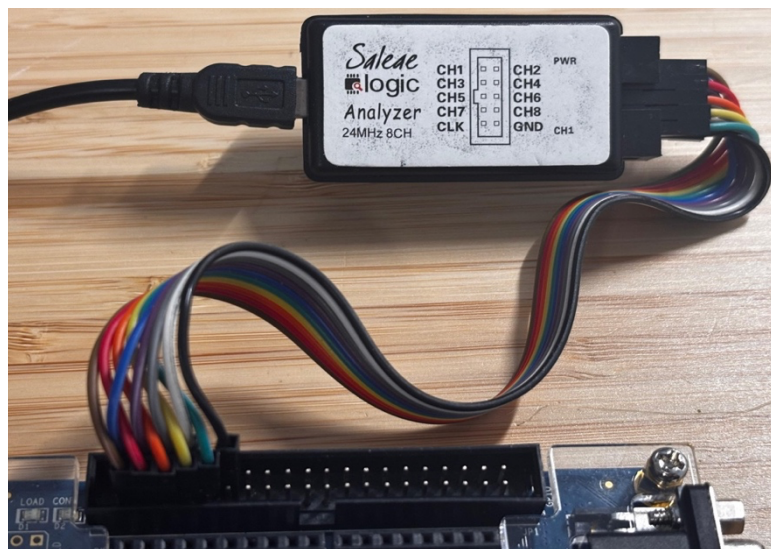


Figure 5 - Saleae Logic Analyzer Connected to the DE10-Lite Board

The logic analyzer provided essential visibility into the system's real time behavior (as seen in chapter 6), allowing us to monitor timing, sequencing, and module coordination that would not have been observable through simulation alone. Using the

Saleae software, we could capture and analyze waveforms with high precision, facilitating debugging and validation of timing assumptions. This tool proved invaluable in confirming correct system operation under actual hardware conditions and complemented both simulation and synthesis by providing physical observability.

This logic analyzer samples signals at up to 24 MHz, and can reliably capture digital transitions at frequencies up to approximately 10 MHz.

4.2 Software Description

The software development and validation of the system relied on a combination of dedicated tools for hardware description, simulation, test generation, and signal analysis. Each tool was selected to fulfill a specific role in the development workflow, from RTL design to runtime debugging.

Quartus Prime Lite Edition

All hardware modules were written in SystemVerilog and developed using Intel Quartus Prime Lite Edition, which served as the primary environment for synthesis, pin assignment, constraint management, and FPGA programming. The full system, including the MAC Core, test generator, validator, control unit, memory, and peripheral interfaces, was implemented and compiled using Quartus.

Quartus also provided tools such as the RTL Viewer and Block Diagram Tool, which were used to inspect module hierarchies, verify signal connectivity, and confirm that synthesis mapped the logic correctly to the FPGA.

ModelSim

To validate the functionality of individual RTL blocks before synthesis, we used ModelSim (Intel Starter Edition) for pre-synthesis simulations. Modules such as the validator, and control unit were simulated independently to verify FSM transitions, timing correctness, and output behavior under various input conditions. This allowed debugging at an early stage, before hardware deployment.

Visual Studio Code

The test vectors used during validation were generated using custom Python scripts. Development of these scripts was done in Visual Studio Code, which provided a streamlined environment for writing and debugging the Python-based test generator. The script generated 500 such tests and exported them into a .sv memory file, which was loaded into the FPGA's on-chip memory before test execution.

Logic 2.4.29 (Saleae)

To observe and debug signal behavior during real-time operation, we used the Logic 2.4.29 software provided by Saleae, in combination with the physical logic analyzer. The software enabled multi-channel signal capture, zooming, annotation, and waveform inspection. It was particularly useful for identifying signal timing issues and ensuring proper synchronization between components.

5 RTL structure and system integration

Following synthesis in Intel Quartus Prime, an RTL-level schematic was generated to represent the physical instantiation and connectivity of all modules in the design. The figure below shows the full system view, including each of the major components and their signal level interconnections.

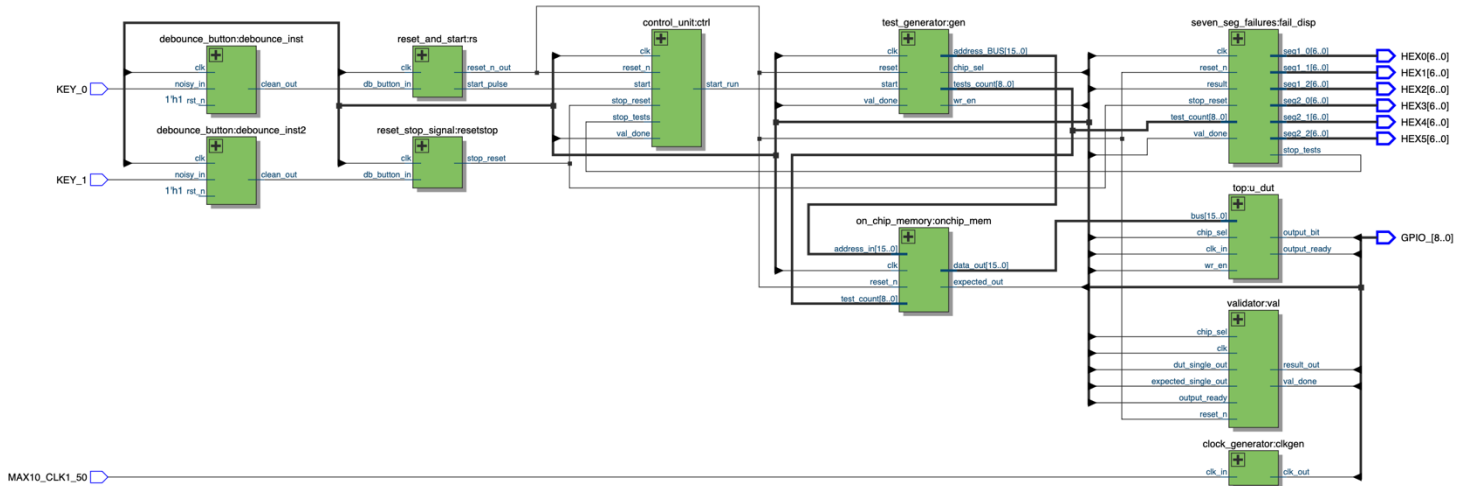


Figure 6 - RTL Block Diagram Generated by Quartus

Each green block corresponds to a synthesizable module defined in the SystemVerilog implementation. These include:

- **Clock_generator:** Generates the internal system clock from the external 50 MHz MAX10 oscillator. All synchronous modules are driven by this clock.
- **Debounce button (×2):** Filters out mechanical bouncing from the physical push buttons KEY0 and KEY1, ensuring clean and stable logic transitions for reset and resume events.
- **Test Generator:** Streams input data to the DUT, including 64 pixel-weight pairs and a 22-bit threshold value for each test. It handles memory addressing and controls the sequencing of data transfer.
- **Control Unit:** Governs the entire system's operation, managing test initiation, response to validation results, halting the system after errors, and coordinating resets and resume conditions.
- **Validator:** Compares the DUT's single-bit output to the expected result stored in memory. If a mismatch is detected, it flags the failure, increments the internal failure counter, and triggers the display logic.
- **On-Chip Memory:** Stores all test data (33,000 words) and expected outputs (500 bits). The use of on-chip memory instead of external SDRAM ensures low-latency access and high timing determinism during validation.
- **Device Under Test (DUT):** Contains the MAC Core and threshold-based activation logic. It receives pixel-weight inputs, accumulates the weighted sum, and applies a binary threshold function.
- **Reset & Resume Mechanisms:** KEY0 (Reset + Start) and KEY1 (Resume Tests) are mapped to physical buttons on the board. These allow the user to initialize the system and resume tests after failure events without reprogramming the FPGA.

- 7-Segment Display Logic: Onboard seven-segment displays (HEX0-HEX5) are used to indicate the index of the first two failing test cases. After two failures, testing halts automatically.
- GPIO Output: Provides access to selected debug signals on external pins for real-time observation via a logic analyzer.

This synthesized representation was used to verify that all module instantiations and signal interfaces matched the intended design. It also served as a valuable cross-check before hardware programming, ensuring correctness at the system integration level.

Connectivity verification was performed using Quartus's RTL Viewer and the Messages panel. These tools allowed us to inspect inter-module connections, verify that all ports were correctly driven, and ensure that no signals were left floating or mismatched. Warnings related to unused or unconnected ports were reviewed and addressed to maintain structural correctness before synthesis. This process helped detect issues such as width mismatches, unconnected outputs, or missing control lines early in the development cycle, reducing the risk of runtime errors and simplifying debugging.

5.1 System Workflow Overview

Before execution begins, a Python script on the host machine generates 500 test cases, each consisting of 64 pixel-weight pairs, a 22-bit threshold, and an expected single-bit result. These are stored as 66 16-bit words per test and compiled into a Memory Initialization File, which is then loaded into the FPGA's on-chip memory. Once this process is complete, all input vectors and expected results reside entirely on the device, eliminating the need for external memory access during runtime.

To start testing, the user presses KEY0 on the DE10-Lite board. This signal is debounced and fed into the reset and start logic, which generates a global reset followed by a one cycle start pulse to the control unit. The control unit then issues a start_run signal to the test generator, initiating the first test. The generator reads 66 words from memory and streams them sequentially to the DUT while asserting chip_sel. For each word, it computes the read address, waits for valid data, and pulses wr_en. This sequence includes 64 pixel-weight pairs and the two threshold words.

As the test data is streamed, the DUT performs a MAC operation and applies a binary threshold to produce a single-bit result. When the computation is complete, it raises output_ready, prompting the validator to capture and compare the DUT output to the expected value from memory. A single-bit result signal is generated, and val_done is asserted for one cycle to notify other blocks that validation is complete. If the result is correct, the system continues seamlessly to the next test. If a mismatch occurs, the failure index is recorded and displayed on the seven-segment displays. After two failures, the system halts automatically and waits for user intervention. Pressing KEY1 triggers a resume mechanism that clears the failure counters and reactivates testing from the point where it stopped.

This cycle repeats until either 500 tests are completed or halted after failures. At the end of execution, the generator and control unit transition to idle states, and the system remains halted until manually reset.

6 Test execution and Validation results

6.1 Test Duration

During continuous test execution, where multiple test cases are run sequentially, each individual test (excluding the first) completes in exactly 75 clock cycles. This measurement starts from the moment the control unit asserts the start signal to the generator, and ends when the val_done signal is raised by the validator, indicating that the result has been captured and processed.

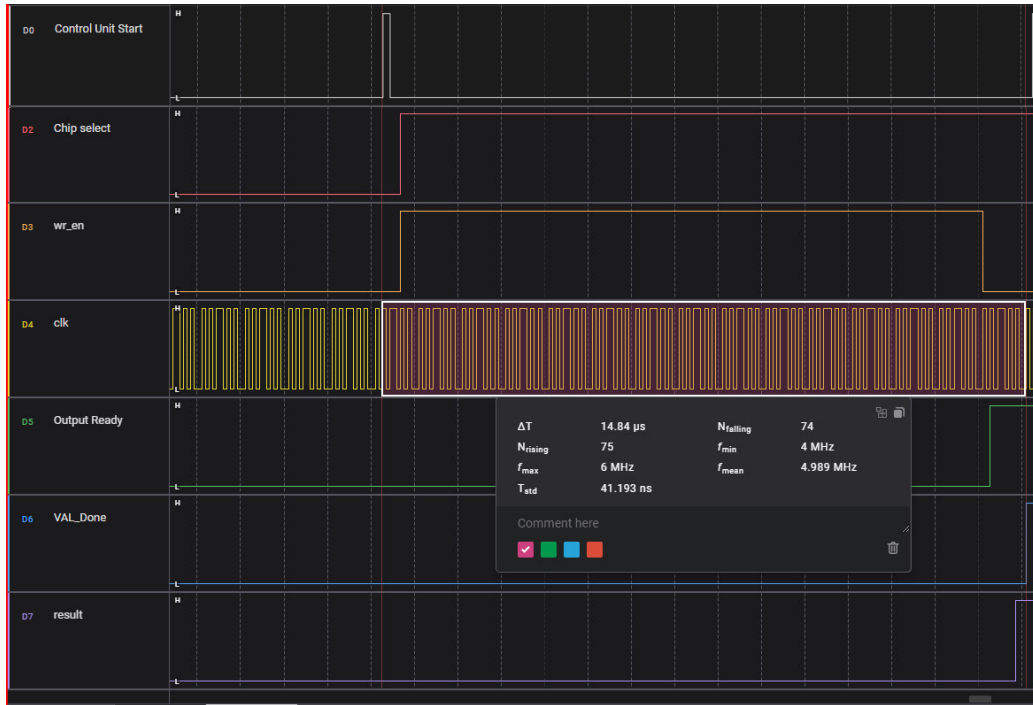


Figure 7 - Single Test Duration in Continuous Execution (Test $n > 1$)

To clearly observe the internal signals, transitions during execution and receive the correct clock cycles, the system clock was slowed down to 6 MHz, ensuring compatibility with the sampling limits of the Saleae Logic Analyzer. Although the analyzer nominally supports up to 10 MHz, we found that stable and complete signal capture required significantly lower frequencies.

First Test Latency

The execution of the first test in the sequence takes slightly longer due to initial system activation. The process begins with the user pressing KEY0, which passes through a debouncing module and triggers the reset and start logic. Only after this part is complete does the control unit issue a start signal to the test generator, initiating the first test.

This adds a short initialization delay before the test itself begins, making the first test to complete in exactly 76 clock cycles. While not affecting the correctness or behavior of the test, it distinguishes the first test from the rest in terms of total execution time.

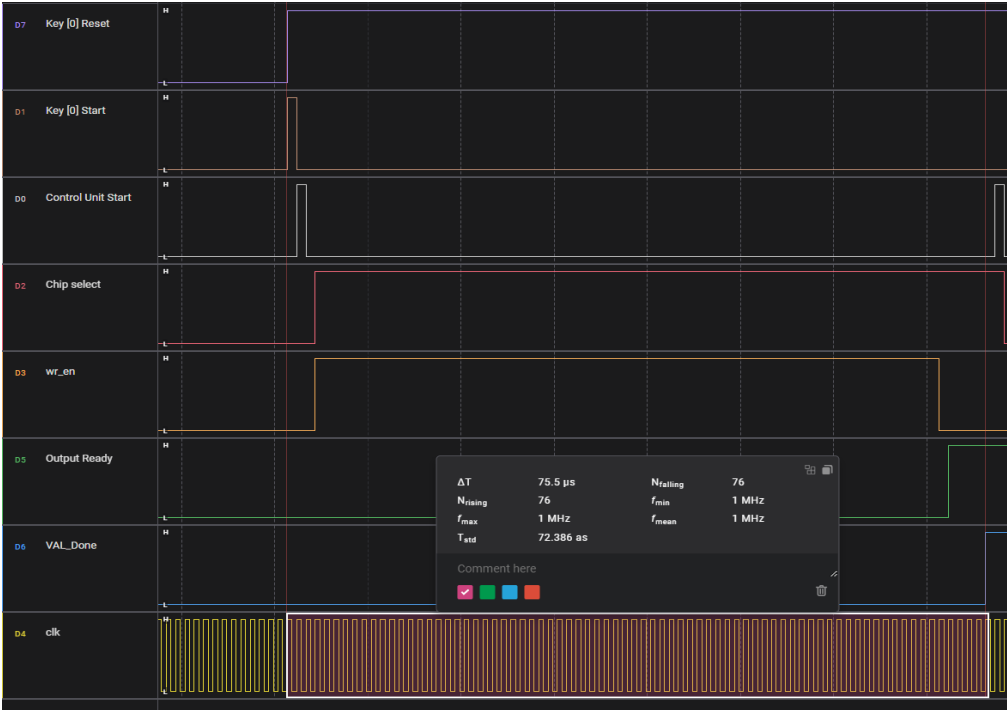


Figure 8 - First Test Execution Timing

Similarly, to clearly observe the internal signals and transitions during execution, the system clock was slowed down to 1 MHz, ensuring compatibility with the sampling limits of the Saleae Logic Analyzer.

6.2 Execution of Multiple Tests

The system was evaluated in full test mode, executing hundreds of tests consecutively without interruption. The figure below shows an excerpt of this process, where you can see 44 tests out of the 500, executed in sequence.

Each pulse on the Val_done signal corresponds to the completion of one test, indicating that the validator has compared the DUT's output with the expected result. Throughout this sequence, the control signals wr_en, chip_sel, and output_ready follow a repeating pattern every 75 cycles, confirming that the test pipeline remains stable and synchronized over time. The DUT_Output signal represents the single-bit binary result produced by the DUT after applying the MAC operation and threshold comparison, while expected_result reflects the corresponding expected output stored in memory for each test. The alignment between these two signals, in the clock cycles where DUT_output_ready is high, across all tests, confirms correctness.

The final result line remains high throughout the capture, indicating that all 44 tests passed successfully without mismatches.

This continuous execution capability, with consistent timing and no stalls, demonstrates the system's robustness and its ability to validate hundreds of tests efficiently and autonomously.

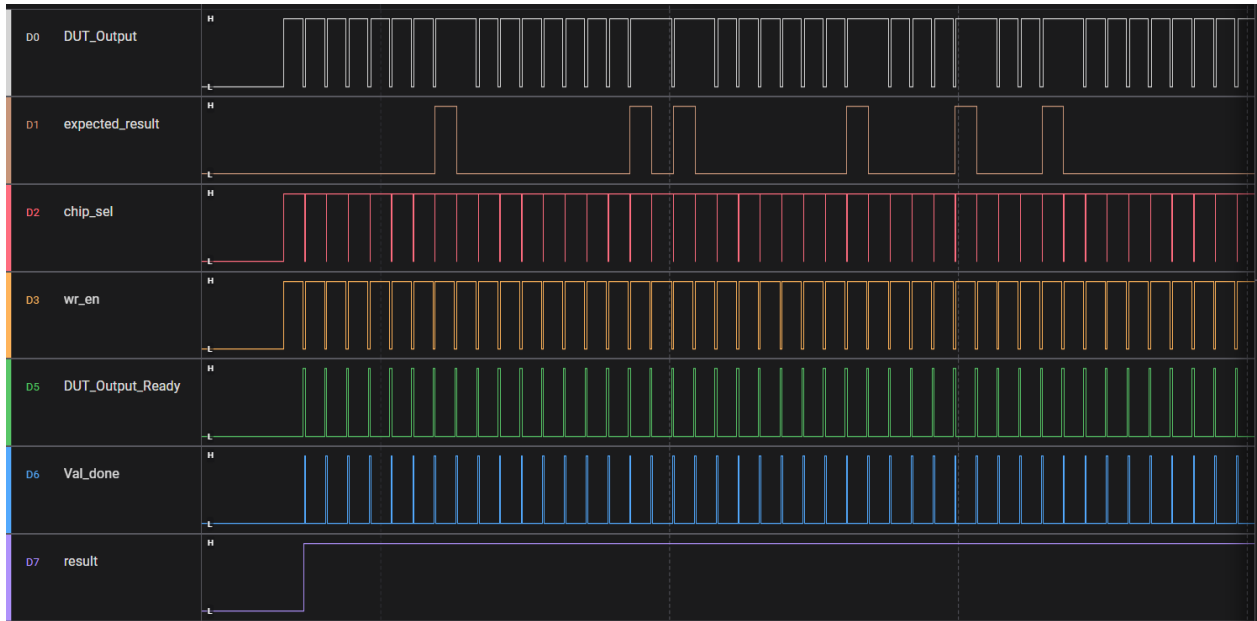


Figure 9 - Logic Analyzer Capture Showing 44 Consecutive Tests

6.3 Failure Indication on 7-Segment Display

When the system detects a mismatch between the DUT output and the expected result, it records the failing test index and presents it on the onboard 7-segment display. The first failure is displayed on the three rightmost digits (HEX0-HEX2), and if a second failure occurs, it is shown on the left (HEX3-HEX5). Once two failures are recorded, the system halts automatically to prevent further test execution.

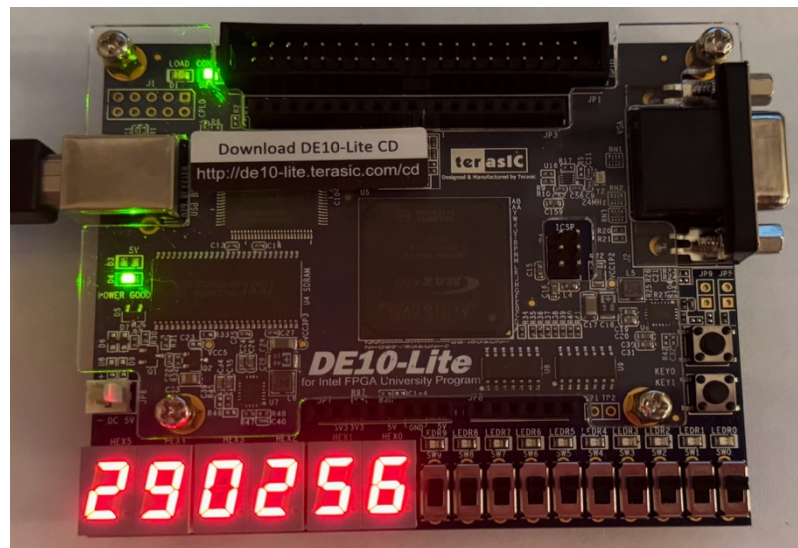


Figure 10 - Failure Display Showing Test Indices 290 and 256

This built-in display mechanism provides immediate visual feedback to the user, eliminating the need for host-side debugging tools to identify at which points tests failed. Testing can be resumed manually via KEY1, which clears the failure state and continues from the next untested index.

6.4 Quantitative Requirement #1: Coverage

6.4.1 Data Coverage

One of the key quantitative requirements of the project was to achieve comprehensive data coverage across the full range of input conditions the MAC Core might encounter in real-world scenarios. This requirement ensures that the system is not only functionally correct under ideal inputs, but also robust against boundary values, irregular input patterns, and asynchronous operational behavior.

As defined in the project test plan, the validation environment was designed to cover both randomized input variation and directed edge cases. In total, around 15000 unique test cases were executed, in batches of 500 tests, each test consisting of 64 pixel-weight pairs (packed into 16-bit words), a 22-bit threshold, and a single-bit expected result. These tests were crafted to simulate realistic and extreme conditions that the DUT must handle reliably.

To maximize data diversity, tests included:

- Random pixel and weight values, covering the full 8-bit range (0-255).
- Thresholds spanning from near-zero to maximum 22-bit values.
- Sparse patterns with mostly zeros and isolated non-zero values.
- Inputs near the decision boundary (cumulative sum $\approx$ threshold).
- Edge cases such as all-zeros and all-ones input vectors.

In addition to value-based diversity, tests were also crafted to cover dynamic operational conditions at the system level. Around 10,000 such tests were executed, targeting scenarios where the input stream is disrupted or delayed, and where control signals are modified mid-test. Such cases test the DUT's response to:

- Partial or paused data transfer
- Loss of chip selection mid-stream
- Delayed validation of results after output_ready

These situations are critical for data coverage, as they expose potential failures in timing assumptions, state retention, or edge-sensitive signal behavior that are not always visible through functional input alone.

While it is infeasible to exhaustively test all possible 66-word input combinations (due to the exponential size of the input space), the executed tests - including both standard and signal variant scenarios, successfully covered 100% of the functional test types relevant for pre-silicon validation of a MAC-based neuron accelerator.

In the following sections, we demonstrate several such tests with logic analyzer captures, showcasing how the system behaves under non-ideal but realistic data-handling scenarios. These examples provide clear evidence that the system not only received varied and complex data patterns, but also remained stable and accurate throughout. Importantly, all of these tests were performed using input files that had already passed standard functional validation, ensuring that any observed failures could be attributed solely to signal timing disruptions or mid-test interruptions.

Delayed Result Validation Test

To further validate the system's ability to handle irregular timing conditions, a randomized delay mechanism was implemented between the DUT's output and the validator's read operation. Instead of reading the DUT output immediately upon output_ready, the validator introduces a variable delay of 0-7 clock cycles before sampling the result. This tests the system's resilience to asynchronous validation conditions and confirms that the DUT holds its output stably across multiple cycles. The output_ready signal (green) is held high for varying durations across tests. This delay corresponds to randomized waiting cycles before the validator samples the result.

In this capture, each test completes successfully, but the timing between output_ready and val_done changes from test to test, confirming that the validator is waiting a random number of cycles before reading the result. This variation ensures that the DUT's output remains valid even when not sampled immediately.

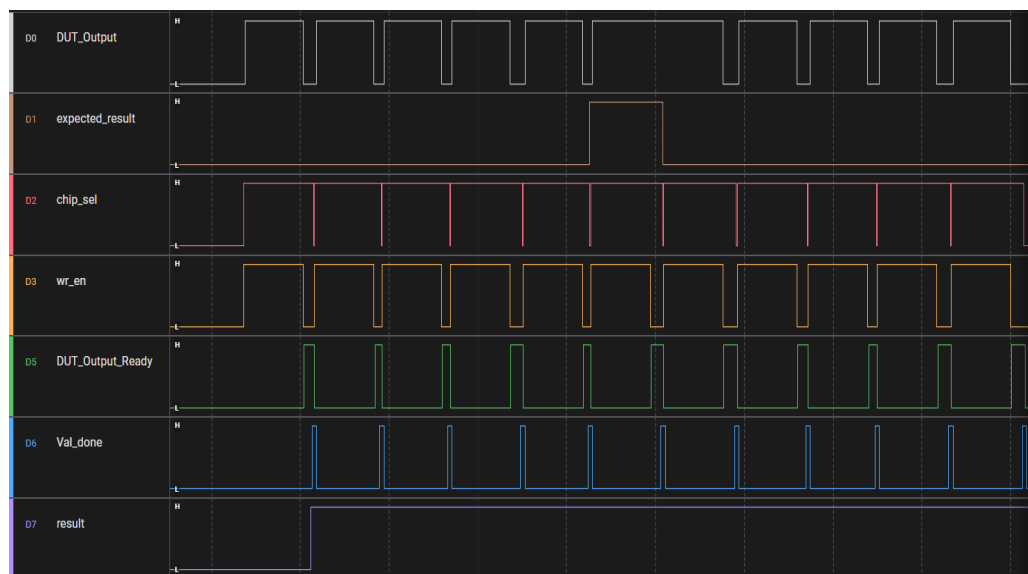


Figure 11- Delayed Result Capture Across Multiple Tests

The validator FSM (VAL_FSM[2:0]) remains in state 1 (VAL_WAIT_RANDOM) for a randomized number of cycles, as seen in the waveform, before proceeding to sample the DUT output. This mechanism helps expose potential issues such as glitches or output metastability, which may not be caught in immediate-read scenarios.

```
// State encoding
typedef enum logic [2:0] {
    VAL_IDLE           = 3'd0,
    VAL_WAIT_RANDOM    = 3'd1,
    VAL_READ_SINGLE_BIT = 3'd2,
    VAL_WRITE_RESULT    = 3'd3,
    VAL_DONE           = 3'd4
} val_state_t;
```

Figure 12 - Validator FSM State Encoding while executing delayed result tests

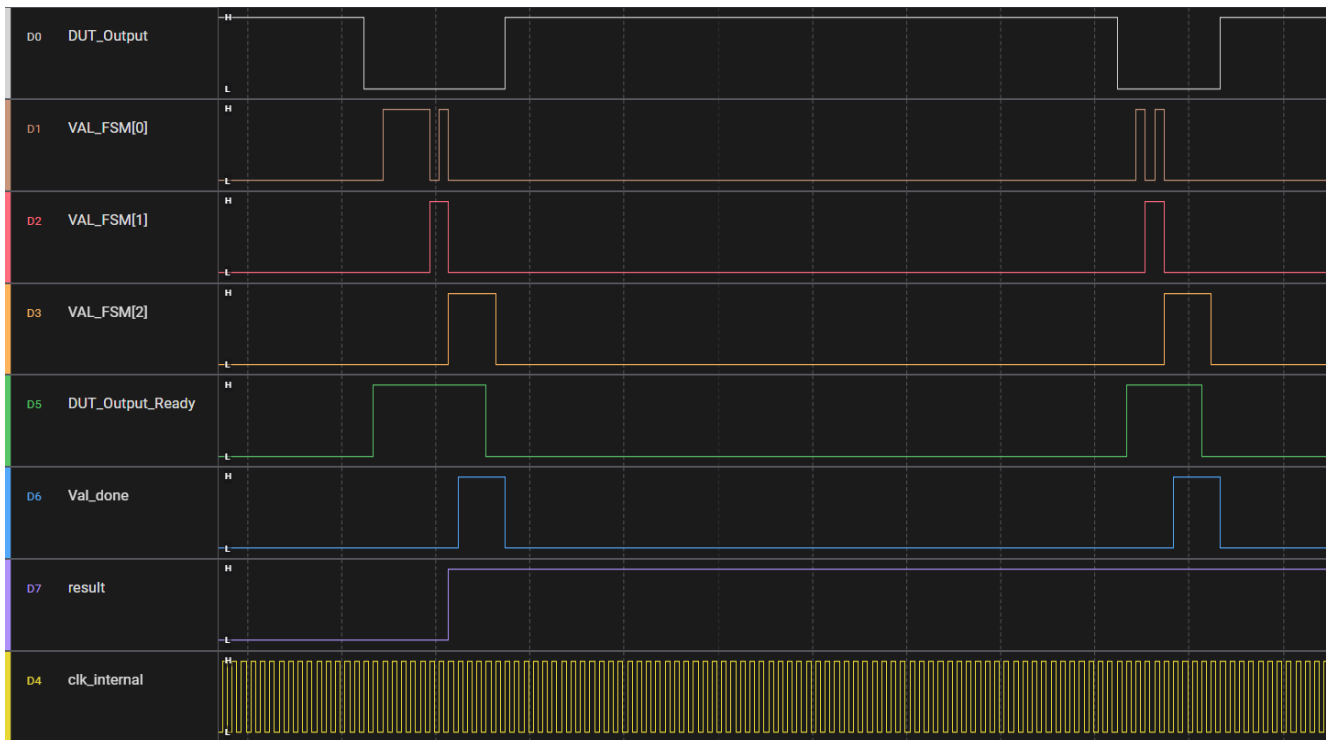


Figure 13 - FSM State Capture in the Validator During Delayed Read

This testing strategy confirms that the DUT and its output interface remain stable across varying validation timings - an essential aspect of robust data handling in asynchronous systems.

Interrupted Data Flow test

Another critical aspect of data coverage is ensuring correct system behavior when the data stream is unintentionally or artificially disrupted. In this test scenario, the `chip_sel` signal, which enables data transfer to the DUT, is intentionally dropped in the middle of an active test. This simulates a real-world case where the test generator may lose synchronization, encounter a stall, or be disabled temporarily due to noise or control errors.

When `chip_sel` is deasserted mid-test and later reasserted, the DUT is designed to reset its internal state completely, discarding any previously received data. This mechanism ensures that partially streamed input does not persist and affect future results. Upon re-enabling `chip_sel`, the DUT treats the following data as the start of a new test, with all accumulation registers and internal buffers cleared.

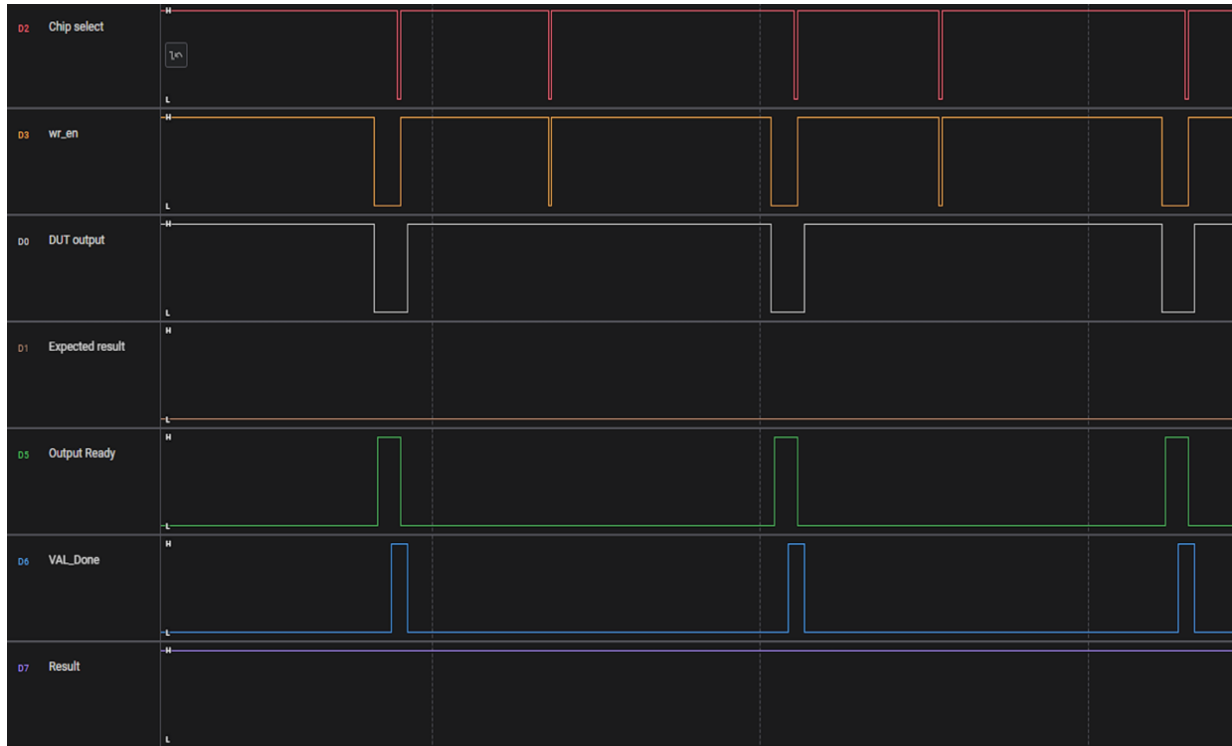


Figure 14 - chip_sel Drop and Correct Test Restart

This behavior was verified by introducing a controlled drop and rise of chip_sel during the streaming of test data. As expected, the interrupted test was effectively ignored, and once the signal was raised again, the DUT processed the subsequent input from scratch. The result matched the expected output in the clock cycles that output ready is high, confirming that no residual or corrupted data remained from the interrupted sequence.

Non Continuous Data Streaming Test

Another scenario examined as part of data coverage was the system's behavior when data is not streamed continuously to the DUT. In this test, the chip_sel signal remains asserted throughout the entire test, but the wr_en signal, which enables the DUT to latch incoming data, is selectively deasserted for random clock cycles during the transmission of the 64 pixel-weight pairs.

This test simulates conditions such as temporary stalls in the data path, fluctuations in control timing, or deliberate pacing variations introduced by the test generator. The goal is to verify that the DUT correctly processes input values only when wr_en is high, and that it does not latch incorrect or repeated data when the signal is low.

During the test, wr_en was randomly held low for a few cycles, causing certain pixel-weight pairs to be delayed. The DUT's MAC unit accumulated only the values received during valid wr_en pulses, and the final output was compared against the expected result that accounted for this irregular input pattern. This verified that the DUT handled input pacing gracefully and produced the correct output based on the actual data it latched.

Due to internal architectural limitations of the DUT, the final two input words, which encode the 22-bit threshold, must be written consecutively in two back-to-back cycles. A gap between these two threshold words may cause incorrect threshold decoding and an invalid result. This constraint was carefully respected during all non-continuous streaming tests.

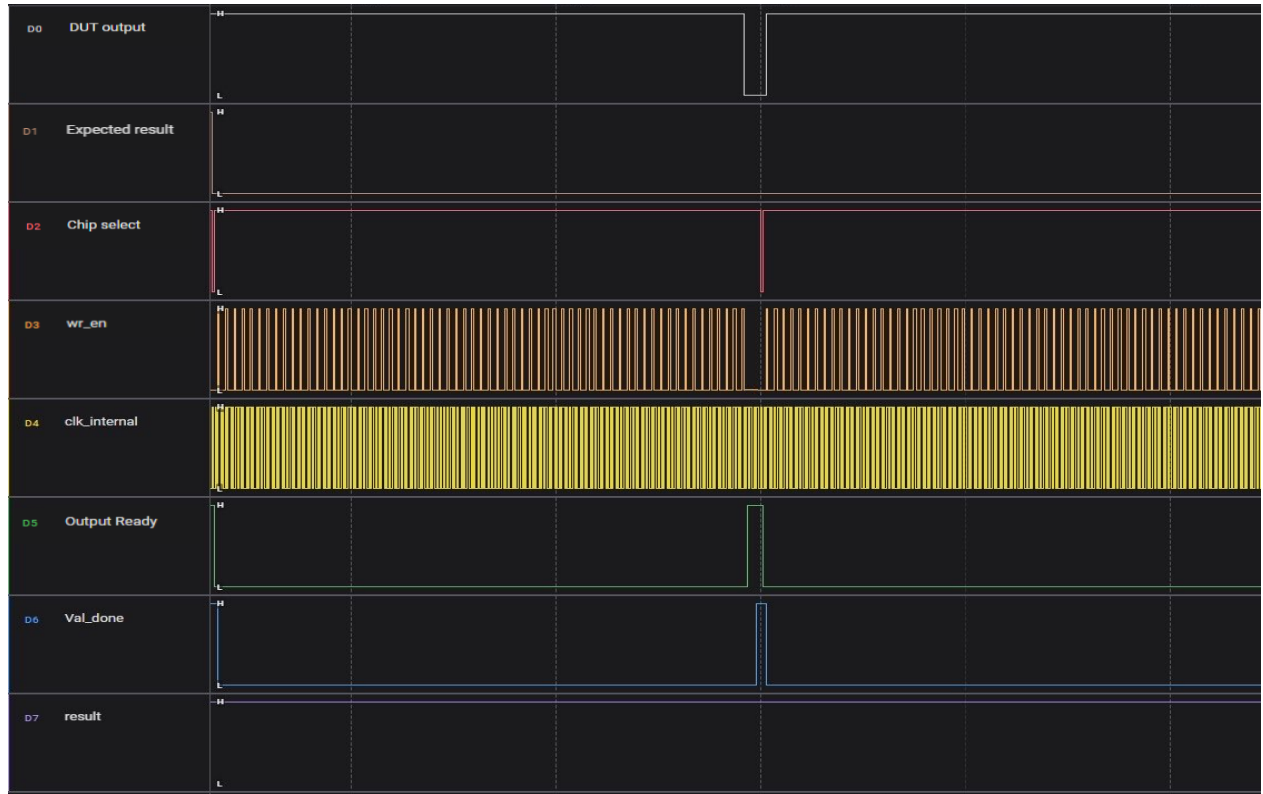


Figure 15 - non-continuous wr_en Pulses During Data Streaming

This test confirmed that the DUT tolerates non-uniform data timing throughout most of the input sequence, as long as the threshold input is delivered atomically at the end. It demonstrates robustness in asynchronous or variable-latency environments, which is common in real-world embedded systems.

6.4.2 FSM coverage

Beyond data and timing variations, one of the core goals of functional validation is to ensure that each Finite State Machine (FSM) in the system traverses all of its defined states during operation. FSM coverage confirms that no part of the control logic remains untested, and that transitions behave as expected in real hardware.

To that end, our objective was to achieve 100% FSM coverage, a realistic and essential goal in a well-structured system with a small number of clearly defined states.

The system contains three major FSMs:

- Control Unit FSM - governs the overall test execution logic.
- Generator FSM - manages streaming of input data from memory.
- Validator FSM - handles DUT output sampling, comparison, and signaling.

To verify FSM coverage, we performed waveform capture during the execution of a single test, starting from the start signal (triggered by KEY0) and ending when the Generator moves to state 3 (finished executing all tests - in this case, one test). During this window, we sampled the state signals of all FSMs and verified that each one passed through all of its defined states.

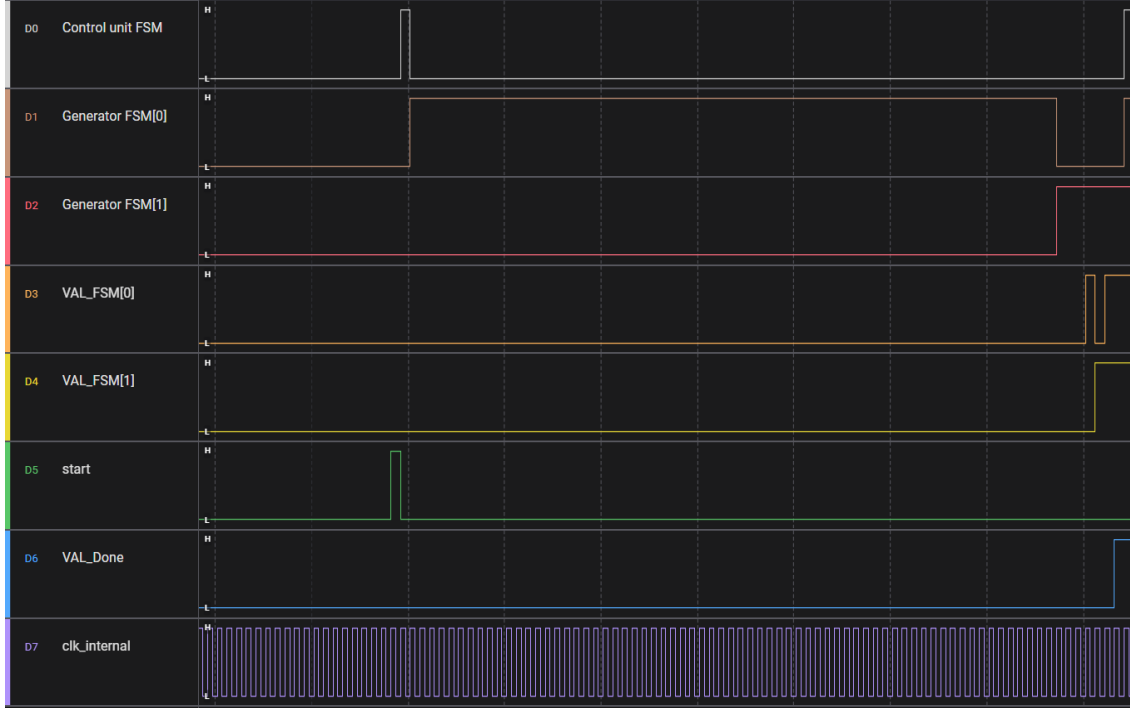


Figure 16 - FSM State Capture During Single Test Execution

```
typedef enum logic [0:0] {
    ST_IDLE = 1'd0, // waiting for start
    ST_START = 1'd1 // issue start_run pulse
} state_t;
```

Figure 17 - Control Unit FSM State Definition

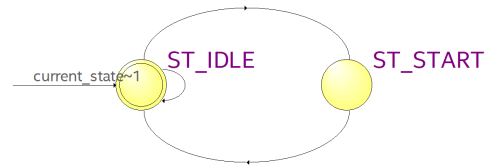


Figure 18 - Control Unit Synthesized FSM Structure

```
typedef enum logic [1:0] {
    IDLE = 2'd0, // waiting for start signal to start next test
    WRITE = 2'd1, // drive address for 66 data lines
    WAIT_VAL = 2'd2, // wait for validator to finish, keep chip select high
    DONE = 2'd3 // all 500 tests are done
} state_t;
```

Figure 19 - Generator FSM State Definition

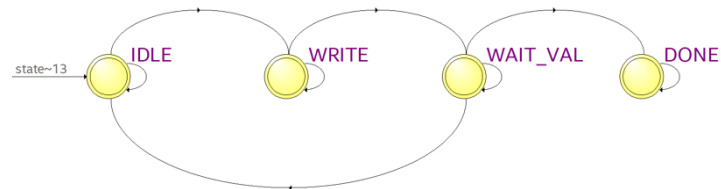


Figure 20 - Generator Synthesized FSM Structure

```
typedef enum logic [1:0] {
    VAL_IDLE = 2'd0, //waiting for output_ready signal to start validation
    VAL_READ_SINGLE_BIT = 2'd1, // latch both sides of the comparison
    VAL_WRITE_RESULT = 2'd2, // compare the two latched values
    VAL_DONE = 2'd3 // holds DONE until output_ready de-asserts
} val_state_t;
```

Figure 21 - Validator FSM State Definition

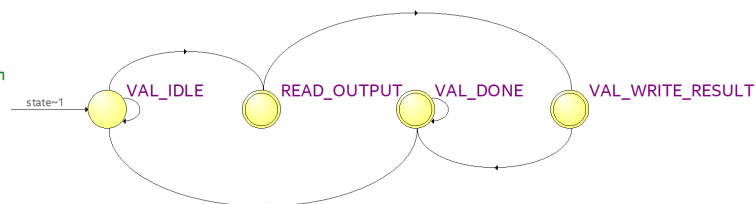


Figure 22 - Validator Synthesized FSM Structure

As seen in the waveforms, the system begins in idle states and, following the start pulse, sequentially progresses through all relevant states for each FSM. This confirms that transitions are correctly triggered and that no state remains unreachable during actual execution.

To complement this behavioral evidence, Quartus-generated FSM diagrams are included next to each state definition. These diagrams, produced post-synthesis, visually confirm that all defined states were successfully synthesized and structurally implemented.

Taken together, the waveform traces and synthesized state machine diagrams validate that FSM coverage reached 100%, ensuring that all sequencing and control logic was fully exercised and verified under real operating conditions.

6.4.3 Toggle coverage

An additional coverage we evaluated is Toggle Coverage, which refers to the percentage of signals in the design that switch states during execution. This metric reflects the dynamic activity across the system and is essential both for validating functional activation across logic blocks and for producing accurate power estimates.

Our goal was to achieve at least 95% toggle coverage, ensuring that all meaningful logic paths in the design are activated during testing.

To evaluate toggle coverage, we compiled the system with a fully populated test memory containing 500 test cases. The Quartus Power Analyzer was then run with vectorless estimation, which analyzes the design's internal activity.

	Data Source	Total	Pin	Registered	Combinational
1	▸ Simulation (from file)				
2					
3	▸ Node, entity or clock assignment				
4					
5	▼ Vectorless estimation				
1	-- Number of signals with Toggle Rate from Vectorless estimation	25358 (99.9%)	52 (83.9%)	717 (100.0%)	24589 (100.0%)
2	-- Number of signals with Zero toggle rate, from Vectorless estimation	15 (0.1%)	0 (0.0%)	4 (0.6%)	11 (0.0%)

Figure 23 - Confidence Metrics from Quartus Power Analyzer showing toggle coverage

As shown in the results, 99.9% of the signals were successfully assigned toggle rate values through vectorless estimation. Only 15 signals (0.1%) were estimated to have zero toggle rate, and no signals fell back to assumed or default toggle values. This high level of toggle coverage confirms that the design was thoroughly exercised under realistic operating conditions, offering strong confidence in both the completeness of the functional validation and the credibility of power estimation results.

6.5 Quantitative Requirement #2: Clock Frequency

The second quantitative requirement defined for the system was meeting a minimum clock frequency of 50 MHz, ensuring the entire design could reliably operate at the fixed 50 MHz clock provided by the DE10-Lite board. This constraint stems from the fact that the board sources its main system clock (MAX10_CLK1_50) directly at 50 MHz, and any design not meeting this timing would risk incorrect behavior in hardware.

After compiling the full design in Quartus Prime, we analyzed the Fmax reports under two process/temperature models:

- Slow 1200mV 0°C Model:

$$F_{max} = 61.07\text{ MHz}$$

Slow 1200mV 0C Model Fmax Summary			
<<Filter>>			
	Fmax	Restricted Fmax	Clock Name
1	61.07 MHz	61.07 MHz	MAX10_CLK1_50

Figure 24 - Quartus Fmax Summary at 0°C (Typical Voltage)

- Slow 1200mV 85°C Model:

$$F_{max} = 55.6\text{ MHz}$$

Slow 1200mV 85C Model Fmax Summary			
<<Filter>>			
	Fmax	Restricted Fmax	Clock Name
1	55.6 MHz	55.6 MHz	MAX10_CLK1_50

Figure 25 - Quartus Fmax Summary at 85°C (Worst-Case Operating Conditions)

In both models, the design meets timing comfortably above the 50 MHz threshold, even under worst-case temperature conditions. This confirms that the system not only functions correctly at the default board frequency, but also maintains timing margin for reliable operation across varying thermal and voltage environments.

This result validates the structural and timing integrity of the design, meeting the project’s second quantitative requirement.

6.6 Quantitative Requirement #3: Power Consumption

The third quantitative requirement defined for the project was a maximum system power consumption of 5W. This limit was set based on the power delivery specifications of the DE10-Lite board and the expected resource utilization of our implemented test system. The 5W ceiling includes both static power (consumed while the FPGA is idle) and dynamic power (during full test execution). Ensuring the system operates within this limit guarantees electrical and thermal safety, supports long-term stability, and aligns with good power-efficiency practices in FPGA-based validation platforms.

Revision Name	FPGA_project
Top-level Entity Name	top_level
Family	MAX 10
Device	10M50DAF484C7G
Power Models	Final
Total Thermal Power Dissipation	272.78 mW
Core Dynamic Thermal Power Dissipation	129.26 mW
Core Static Thermal Power Dissipation	90.75 mW
I/O Thermal Power Dissipation	52.78 mW

Figure 26 - Power Analyzer report showing total power dissipation

The results, indicate a total thermal power dissipation of 272.78 *mW*, well below the 5W ceiling. This total includes:

- Core Dynamic Power: 129.26 *mW*
- Core Static Power: 90.75 *mW*
- I/O Thermal Power: 52.78 *mW*

These results confirm that the entire validation platform, including all generators, memory interfaces, and the DUT interaction logic, remains comfortably within the defined power budget under full operational conditions.

7 Post silicon Validation Planning

As part of the preparations for post-silicon validation that will take place next year, we began by selecting an appropriate packaging format for the fabricated MAC Core. The chip will be manufactured in a QFP¹ (Quad Flat Package) configuration, a choice made for its mechanical robustness, accessibility, and compatibility with standard surface-mount assembly processes. The selected variant includes 44 leads with a 0.8 mm pitch, providing sufficient pin count for the full 16-bit data bus, control lines, and power domains. Its symmetrical pinout across four edges simplifies signal routing and allows for compact, testable PCB layouts. The exposed thermal pad and grounded leads also help improve thermal management and signal integrity during operation.

This packaging decision balances integration density and test accessibility, making it ideal for the post-silicon validation phase, where direct probing and debugging access are often required. Moreover, it aligns well with the validation board's layout and mechanical constraints.

Once the packaging and pinout were finalized, we coordinated with the chip design team to receive the official pin placement, ensuring alignment with the FPGA environment. Based on this, we designed a system-level schematic to illustrate how the DUT will physically interface with the FPGA validation setup.

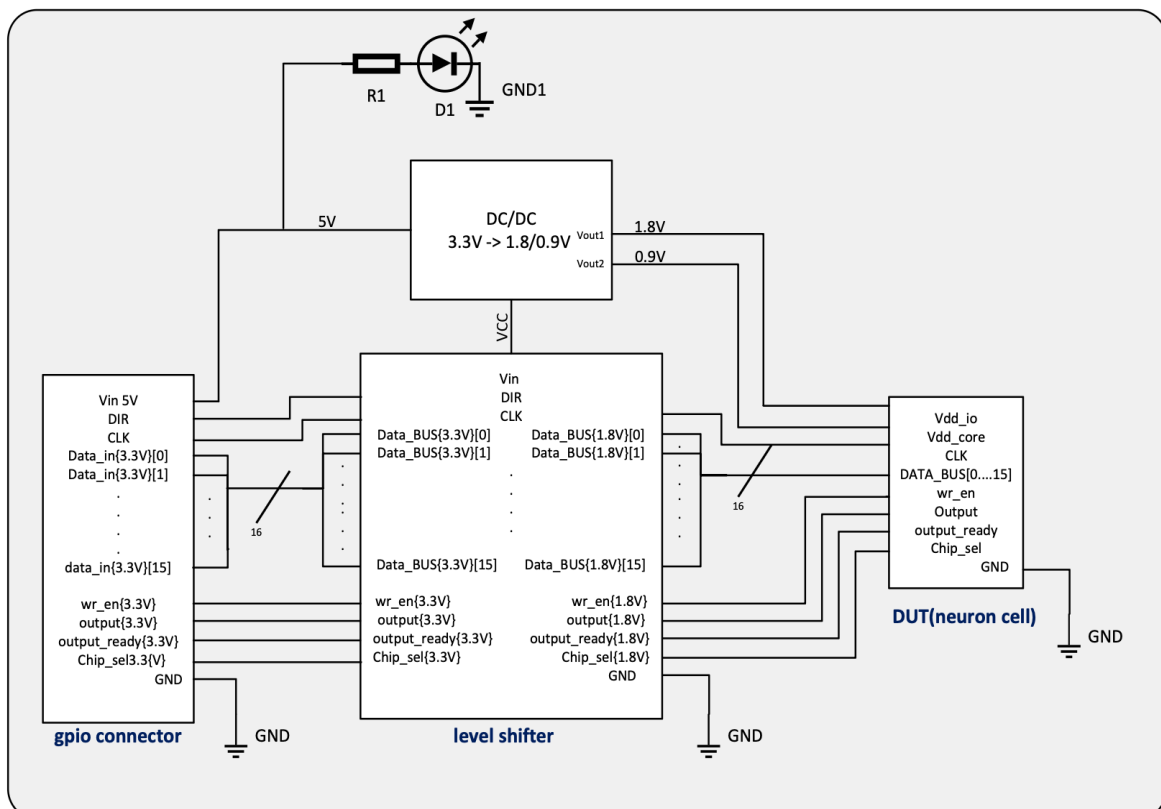


Figure 27 - Physical Integration Schematic for Post-Silicon Validation

¹https://www.ti.com/lit/ml/mpqf075/mpqf075.pdf?ts=1744293486334&ref_url=https%253A%252F%252Fwww.ti.com%252Fpackaging%252Fdocs%252Fsearchtipackages.tsp%253FpackageName%253DQFP

The schematic shows the DUT (on the right), which operates in two separate voltage domains: 1.8 V for I/O (V_{dd_io}) and 0.9 V for core logic (V_{dd_core}). All signals pass through a bidirectional level shifter to ensure compatibility between the 3.3 V FPGA logic and the lower-voltage chip interfaces.

The level shifter is powered by the same voltage rails as the DUT and routes all data and control lines accordingly. On the left, the GPIO connector from the FPGA board provides all input signals, including data, clock, and control, and also receives output signals back from the DUT after level translation. Power is supplied by a DC/DC converter that receives 5 V input and delivers two regulated outputs: 1.8 V for the I/O domain and 0.9 V for the core logic.

This schematic was designed to ensure safe and reliable physical integration between the fabricated chip and the FPGA-based validation system. It establishes a clear interface for future test development and ensures that no electrical mismatches will occur during operation.

With this preparation, the system is fully ready for post-silicon validation, and the FPGA environment developed in this project will be reused by the next team to test the chip under real operating conditions. The modularity of the validation system, together with this integration plan, ensures a smooth transition from simulation to silicon.

8 Evaluation and future work

The primary objective of this project was to design and implement a full validation environment for a custom MAC Core accelerator, enabling reliable pre-silicon verification before tape-out. This goal was successfully achieved.

Throughout the project, the system was tested across a wide range of scenarios, covering randomized inputs, edge cases, control flow variations, and asynchronous behaviors. A total of ≈ 15000 unique tests were executed, each processed entirely within the FPGA without the need for external memory. The validation system detected mismatches in real time, provided immediate visual feedback via 7-segment displays, and included mechanisms for recovery and continuation.

The thorough coverage - including data, FSM, and control signal behaviors, ensured that the MAC Core was tested under diverse and realistic conditions. As a result, confidence in the DUT's functional correctness and robustness was significantly strengthened.

The system's successful operation at a 50 MHz clock frequency, while remaining within the 5W power envelope, confirms that the design meets both timing and resource constraints. These outcomes indicate that the MAC Core is ready for tape-out and fabrication, having been validated thoroughly in a realistic hardware environment prior to silicon production.

8.1 Difficulties and Challenges

While the project ultimately met its goals, several significant challenges arose along the way that shaped both the development process and our learning experience.

One of the most difficult aspects was determining whether the validation system itself was functioning correctly. It is not enough to build a test environment that produces consistent results, we had to ensure that it could actually detect failures when they occurred. This required deep internal debugging and the deliberate injection of faulty outputs to confirm that errors were being caught and reported. Verifying the correctness of the testbench, rather than just the DUT, is a subtle and often overlooked challenge that requires iterative refinement and time.

Another central challenge was the need to work in close collaboration with a separate team responsible for the MAC Core (DUT). Effective communication and technical alignment were essential, yet not always easy to maintain. System-level constraints imposed by one team often required compromises from the other. For example, the DUT team had to adapt some of their interface requirements due to FPGA board limitations, while we, in turn, had to adapt our test environment to the DUT's pinout, functional constraints, and interface assumptions. These changes occurred continuously throughout the project, meaning that a "final version" of the system didn't truly exist until the very end.

Working within the constraints of a real hardware platform (the DE10-Lite board) also posted its own difficulties. While the board offers a rich set of peripherals and debugging options, learning how to effectively leverage these resources, and when to let go of unnecessary complexity, took time. We explored using SDRAM, LEDs, GPIOs, and other features for debugging and memory interfacing, but not all of these efforts were successful or practical for our use case.

Perhaps the most significant technical setback was our original plan to use the onboard SDRAM as the system's primary test memory. We invested substantial effort into building a complete memory interface pipeline using a vendor-supplied VHDL controller, along with a host-based test loader using the Intel Control Panel. However, late-stage issues with memory timing, data reliability, and integration complexity forced us to abandon this approach, despite having worked on it for two full semesters. The decision to switch to on-chip memory was difficult, but necessary given time constraints and the limited external support available for SDRAM debugging. It highlighted the limits of working under academic timelines and with incomplete documentation, as well as the importance of making strategic trade-offs when needed.

8.2 Suggestions for Improvement and Future Work

While the current system fully meets its functional and structural goals, there are several meaningful directions for improvement and future development.

The most prominent opportunity lies in completing the SDRAM-based test memory implementation, which was developed in parallel but ultimately not used in the final project due to integration and timing challenges. The full implementation, including the custom SDRAM interface and validation reporting logic, is preserved in the project's Git repository along with its corresponding architectural documentation. A future team could revisit this design, carefully analyze the challenges we encountered, such as the volatility of SDRAM (which erases data after FPGA reprogramming), and find ways to address them. For example, avoiding reconfiguration between test loads or integrating a stable host-FPGA transfer mechanism could make SDRAM usage feasible. If successful, this approach would allow significantly larger test sets to be executed, improving efficiency and scalability.

Another promising direction for future development is extending the current validation environment to support multi-chip testing. While the present system targets a single MAC Core instance, the broader vision involves deploying multiple such cores on the same board or across different boards. Supporting this functionality would require expanding the control logic to manage multiple validation threads in parallel, modifying the memory architecture to distinguish between test vectors and results for each DUT, and adapting the reporting infrastructure accordingly.

Another possible enhancement is to transform the environment into a fully automated test system, eliminating the need for manual button presses (e.g., KEY0 for reset/start and KEY1 for resume). This could be achieved by integrating a host-controlled trigger or using a continuous test-mode configuration that cycles through test cases without user intervention.

Lastly, this environment is intended to serve not only for pre-silicon validation, but also for post-silicon testing once the MAC Core is fabricated. A future team will be responsible for this phase, using the same validation infrastructure to compare real silicon behavior against the expected functional results. Having a reusable, modular, and well-documented test platform ensures that this transition will be smooth and efficient.

8.3 Lessons Learned

Throughout the course of this project, we gained both technical and professional insights that extended far beyond the original scope of our academic coursework. One of the most significant learning experiences was working hands-on with an FPGA development board. Prior to this project, we had limited exposure to real-world FPGA design and implementation. This project gave us the opportunity to explore how digital systems are physically deployed, debugged, and optimized using hardware platforms like the DE10-Lite.

Another major takeaway was the opportunity to collaborate directly with industry. The project was conducted in partnership with Apple, and through this engagement we were exposed to the expectations, standards, and workflows that guide real chip development cycles. We gained a deeper understanding of the end-to-end silicon development process, including the critical role of pre-silicon validation in reducing risk and ensuring correctness before fabrication.

We also learned that sometimes, in pursuit of the final goal, it's necessary to let go of previous work, no matter how much effort was invested in it. A key turning point in the project was when we decided to abandon our SDRAM-based validation architecture, despite having invested significant time into it. Recognizing that it would not meet timing and stability requirements within our time constraints, we pivoted toward a more reliable and robust on-chip memory solution. This shift taught us to prioritize results over sunk effort, and to remain flexible in the face of design challenges.

Finally, we had the unique opportunity to collaborate with engineers from industry, who supported us throughout the process. Their guidance in debugging, architectural decision-making, and validation strategies contributed significantly to the success of the project and gave us a glimpse into the collaborative nature of real engineering work.

9 Project Documentation

All project files and documentation are hosted in the following GitHub repository:

<https://github.com/BinatM/Neural-Engine/tree/FPGA>

The repository includes the following major components:

Final System Implementation-

- test_environment: This directory includes the full SystemVerilog-based test infrastructure used for running 500 tests in hardware.
- testgen: Contains Python scripts for generating functional and randomized test sets. The output test files follow the expected binary format and are used as inputs to the validation system. The folder also includes multiple subfolders with different generated test sets.
- Project files : Contains .sdc file for timing constraints and .qsf file with Quartus project configuration, including device settings, pin assignments and included source files
- Documents:
 - FPGA Architecture.docx: Full documentation of the hardware architecture and design decisions.
 - Test plan.docx: Describes the verification strategy, test coverage, and goals.
 - Test Execution Summary.docx: Summarizes the actual execution of all tests, including results.
 - README.md: A user guide explaining how to compile and run the system.

Debugging and Single-Test Validation

- single_test_test_environment: A compact version of the system used for initial bring-up and debug of single test vectors. This was used to validate core logic before expanding to 500 test batches.
- testbenches: ModelSim testbenches designed for simulating the single-test version in isolation. These helped verify the validator and DUT behavior before system-level integration.

SDRAM-Based Implementation

- SDRAM Backup (SDRAM_code_backup): A complete backup of the original implementation designed to operate with the DE10-Lite's SDRAM. This version included a custom SDRAM controller and .sv files . There is also an inner folder, containing all doc files for this implementation (architecture, managing SDRAM memory and testing pipeline).
- Validation Scripts (validation): Python scripts for parsing SDRAM output and generating post-test result summaries. These were originally intended for validating SDRAM-based test execution.
- Binary Test Archive (all_tests_bin.zip): A ZIP file containing full SDRAM-compatible test sets, formatted for bulk loading and analysis using the scripts above.

10 References

Datasheet

- [1] Texas Instruments, “CDCE937 Programmable Clock Synthesizer Datasheet”, SLAS564I - August 2007; revised December 2024.
<https://www.ti.com/lit/ds/symlink/cdce937.pdf>
- [2] ISSI, “42/45R and 86400D/16320D/32160D SDRAM Datasheet”, February 2025.
https://www.issi.com/WW/pdf/42-45R-S_86400D-16320D-32160D.pdf

User's Guide:

- [3] Terasic, “DE10-Lite User Manual”, September 2017.
https://ftp.intel.com/Public/Pub/fpgaup/pub/Intel_Material/Boards/DE10-Lite/DE10_Lite_User_Manual.pdf
- [4] Intel, “MAX 10 FPGA Device Handbook”, March 2025
<https://www.intel.com/programmable/technical-pdfs/max10-handbook.pdf>
- [5] Intel, “PowerPlay Power Analyzer Tool User Guide”, December 2019
<https://www.intel.com/programmable/technical-pdfs/683305.pdf>
- [6] Intel, FPGA Design Software User Guides.
<https://www.intel.com/content/www/us/en/support/programmable/support-resources/design-software/user-guides.html>

Other References:

- [7] ChipVerify, Code Coverage in SystemVerilog, accessed 2025 (online article)
<https://www.chipverify.com/verification/code-coverage>